

WHITEPAPER

SOA vs Microservices

Two ways of building software with services, and how to choose between them

Date: August 2018

Audience: Architects, senior engineers and technology decision-makers

Author: Amit Kumar Jha

In one sentence

SOA is a way to connect many different systems across a whole company using shared services and a central 'hub'. Microservices is a way to build one application out of many small, independent parts that separate teams can change and release on their own.

Abstract

Service-Oriented Architecture (SOA) and Microservices are two popular ways to split software into services. They come from the same ideas, but they have grown into two clearly different approaches. This paper explains both, shows where they differ, describes the tools in use today, and gives a clear guide on when to use each one. It also explains when you should use neither and simply build one well-organised application instead.

Key Takeaways

- **The difference is not size.** It is about scope and how tightly things are connected. SOA works across a whole company. Microservices work inside one application.
- **SOA puts the 'smart' logic in the middle.** A central hub (the Enterprise Service Bus, or ESB) translates, routes and controls messages. Microservices put the smart logic inside each service and keep the network simple.
- **Microservices need each service to own its own data.** If services share one database, you get the cost of microservices without the benefit.
- **Microservices need mature operations.** Containers, automated deployment, central logs and tracing are not optional. The tools now exist, but the team still has to learn them.
- **Start simple.** If your team is small or your business domain is still unclear, build one well-structured application first and split it later.
- **Most companies will use both.** An integration layer at the edge of old systems, and microservices inside new products, is a common and sensible mix.

1. Introduction: Why This Question Matters

Every software team eventually faces the same problem. One big program becomes hard to change. Many teams step on each other's toes. A small fix in one place breaks something else. The natural answer is to break the software into smaller pieces called services. The hard question is how to break it up and how the pieces should talk to each other.

SOA and Microservices are two answers to that question. Because they share vocabulary, people often mix them up or argue about which is 'better'. This paper takes a different view: they solve different problems, and the right choice depends on your situation.

2. A Short History

2.1 SOA: connecting the enterprise (about 2000 to 2012)

In the early 2000s, large companies had many separate systems: finance, sales, billing, customer records, old mainframes. Each system was connected to the others with custom code, one link at a time. This became a tangle that nobody could manage.

SOA was the response. The idea was to expose each business capability as a service with a clear contract, put all the services in a catalogue, and connect them through a central hub called the Enterprise Service Bus (ESB). The ESB translated between formats, routed messages, applied security rules and ran multi-step business processes. Standards such as SOAP, WSDL and the WS-* family described how services should talk. Vendors including IBM, Oracle, TIBCO, Software AG, MuleSoft and WSO2 sold ESB products.

Over time, SOA earned a reputation for being heavy. Governance boards slowed changes. The ESB became a bottleneck because every change had to pass through it. A 2009 article titled 'SOA is Dead; Long Live Services' summed up the mood: the ideas were good, but the way they were delivered had become a burden.

2.2 Microservices: fast product teams in the cloud (2011 to today)

Around 2011, a group of architects began using the word 'microservices' for a new style they were seeing at companies like Amazon and Netflix. James Lewis and Martin Fowler wrote the most-read description in 2014. Sam Newman's book *Building Microservices* followed in 2015.

The idea was simple: build one application as a set of small services, each owned by one small team, each with its own data, each released on its own schedule. Docker containers (2013) made it cheap to run many small services. The DevOps movement gave teams the habit of owning their code in production.

The tools have now settled down. Kubernetes has become the standard way to run containers; version 1.11 shipped in June, and all three major clouds offer it as a managed service. The 'service mesh' idea, which handles security, retries and monitoring between services, has become real: Istio released version 1.0 at the end of July, and Linkerd and Envoy are already in production use. Spring Boot 2.0 (March) and Spring Cloud Finchley (June) are the most common Java tools. Apache Kafka, which reached version 2.0 last month, is the usual choice for sending events between services. gRPC is growing for fast internal calls. Serverless functions such as AWS Lambda are being talked about as the next step.

On the SOA side, Salesforce completed its purchase of MuleSoft in May this year. This shows that integration platforms still matter, but they are now sold as 'API-led connectivity' rather than as classic ESBs.

3. What Each One Is

3.1 SOA

Think of SOA as a company-wide postal service. Every department offers a few well-defined services, such as 'look up a customer' or 'create an order'. A central post office (the ESB) receives every request, translates it into a form the receiving department understands, checks that the sender is allowed, and delivers it. The main goals are reuse (build a service once, use it everywhere) and consistency (everyone sees the same customer record). Data is often shared through one common data model. Rules are set centrally.

3.2 Microservices

Think of microservices as a row of small independent shops on one street. Each shop does one thing well, keeps its own stock and its own books, and can open, close or renovate without asking the others. Customers walk directly to the shop they need. There is no central post office; the street itself is simple. The main goals are independence (each team can change and release on its own) and speed. Each service owns its own data. Rules are kept light and are enforced mostly by automation.

3.3 What they have in common

Both styles want services that are loosely connected, have clear contracts, are easy to find, and can be combined. Martin Fowler has described microservices as 'a subset of SOA', meaning one particular way to do SOA well. Others say the differences in team structure and data ownership are big enough to call

them separate styles. For the purpose of making decisions, it helps to treat them as two different things.

4. Side-by-Side Comparison

The table below shows the main differences as they are commonly understood today.

Area	SOA (classic, ESB-based)	Microservices
Scope	The whole company. Connects many applications.	One application or product. Splits it into parts.
Size of a service	Large. One service covers a whole business area, such as 'Customer'.	Small. One service covers one job, such as 'send a welcome email'.
Where the logic sits	In the middle. The ESB translates, routes and controls.	At the edges. Each service handles its own logic. The network stays simple.
How services talk	SOAP and XML messages, message queues, multi-step processes run by the hub.	Simple HTTP (REST/JSON), gRPC, or events through Kafka or RabbitMQ.
Who owns the data	Data is often shared. One common data model for the company.	Each service owns its own database. Nobody else touches it.
Keeping data in step	Distributed transactions make everything update together.	Updates spread through events. Things become consistent a little later.
Reuse	Very important. Services are built for many users.	Less important. Some duplication is accepted to keep teams independent.
Rules and control	Central. Registries, review boards, approved data formats.	Team-owned. Automated contract tests and a few shared standards.
Releasing changes	Services often released together on shared servers.	Each service released on its own, usually in a container.
Team shape	Separate teams for integration, database and operations.	One small team builds and runs its own services.
Technology choice	Usually one vendor platform, one language.	Teams may pick different languages within agreed limits.
Handling failures	The hub retries and guarantees delivery.	Each service uses time-outs, retries and 'circuit breakers'. Service mesh is emerging.
Seeing what happens	Monitor the hub. Following one request across systems is hard.	Central logs, metrics and request tracing (Zipkin, Jaeger, Prometheus).
Common failure	The hub becomes a bottleneck and a single point of failure.	'Distributed monolith': many services that are still tightly tied together.
Typical tools today	MuleSoft, IBM Integration Bus, Oracle SOA Suite, TIBCO, WSO2, webMethods.	Kubernetes, Docker, Istio, Linkerd, Spring Boot, Spring Cloud, Netflix OSS, Kafka, gRPC, AWS Lambda.

5. The Differences That Matter Most

5.1 Smart middle or smart edges?

This is the biggest difference. In SOA the hub is deliberately clever. That gives consistency and lets old systems join without being changed. But every new business flow means a change to the hub, and the hub team becomes a queue that everyone waits in.

Microservices move the cleverness into each service. A team can change its service from end to end without asking anyone. The price is that common needs such as security checks, retries and tracing must be handled in every service. This is exactly the gap that the service mesh has been built to fill over the last two years: a small helper process next to each service takes care of those common needs, without bringing back a central hub for business logic.

5.2 Who owns the data?

SOA is comfortable with shared databases and one company-wide data model. That works when data changes slowly. Microservices insist that each service keeps its data private. This is what makes independent releases possible, but it forces you to accept that data will be consistent 'eventually' rather than instantly, and to learn patterns such as Saga (a chain of steps with undo actions) and event-driven updates.

This is the most under-estimated cost of microservices. A team that splits its code into services but keeps one shared database has built a 'distributed monolith': all the complexity of many services, none of the freedom.

5.3 How small is 'micro'?

The right size is not measured in lines of code. A microservice should match one clear business area (a 'bounded context') and be owned by one team. A useful rule of thumb: a team should be able to rewrite its service in about two weeks. Make services too small and the network becomes the problem; this is called the 'nanoservice' mistake. SOA services, in contrast, are large on purpose because they serve many users for many years.

5.4 Rules and people

SOA relies on formal governance: registries, design authorities, approval boards. That suits companies where regulation or company-wide consistency comes first. Microservices replace most of this with automation and agreed habits: automated contract tests between services, a simple versioning rule, and a short list of things every service must do (health check, standard log format, tracing header). The hidden requirement is trust: teams must be allowed to run their own services in production. Without that, microservices just recreate the old silos with more moving parts. This is Conway's Law in action: the shape of the software will follow the shape of the organisation.

5.5 The cost of running it

Martin Fowler called it the 'microservice premium': extra cost for infrastructure, monitoring, testing and debugging. It pays back only when the application is large enough that one big program would cost more in coordination. Today the premium is much lower than it was in 2015, thanks to managed Kubernetes on every major cloud, Istio 1.0 and a mature Spring Cloud. But it has not disappeared. SOA has its own premium: licence fees, specialist skills for the integration platform, and the slow pace of central change control.

6. When to Use Which

6.1 Choose SOA (or an integration platform) when

- Your main job is connecting many old or packaged systems (ERP, mainframe, off-the-shelf software) that cannot be changed and speak different protocols.
- The business needs one governed, company-wide view of key records, for example to meet regulatory reporting duties.
- Business processes must run reliable multi-step transactions across systems, and 'undo' steps would be impractical.
- The company is organised into functional teams with central change control, and is not ready to reorganise around products.
- Change is slow or moderate, and a central bottleneck costs less than inconsistent data would.

6.2 Choose Microservices when

- You are building or growing a product whose business areas are clear, and different parts change at different speeds.
- Releasing quickly and independently is the top priority, and several teams must ship without waiting for each other.
- Different parts have very different needs for scale, uptime or technology (for example, a heavy recommendation engine next to a simple catalogue).
- You already have, or are committed to building, cloud-native operations: containers, automated pipelines, central logging, tracing, and teams who are on call for their own services.
- You can live with data being consistent a little later, and design for it with events and repeat-safe operations.

6.3 Build one well-organised application first when

- The team is small (roughly fewer than two full product teams) or the business areas are still unclear. Fowler's 'Monolith First' advice remains the most quoted guidance on this point: splitting too early locks in the wrong boundaries.
- Operations maturity is low. A single application with clean internal modules can be split later along the lines that prove real.

6.4 Decision matrix

Your situation	SOA	Microservices	Single modular app
Connecting old and packaged systems	Strong fit	Weak fit (wrap them behind a translation layer)	Not relevant
Many teams releasing fast and independently	Weak fit	Strong fit	Moderate (one release unit)
Company-wide reuse and one data model	Strong fit	Avoided on purpose	Not relevant
Everything must update together	Strong fit	Weak fit (needs Saga and undo steps)	Strong fit
Parts need very different scaling	Limited	Strong fit	Weak fit

Your situation	SOA	Microservices	Single modular app
Small team, new or unclear domain	Too heavy	Too heavy	Strong fit
Low DevOps or cloud experience	Workable with a vendor platform	High risk	Strong fit
Need several languages or stacks	Limited	Strong fit	Limited
Strict central governance or audit	Strong fit	Possible with platform guard-rails	Strong fit

6.5 Using both, and moving from one to the other

For most companies the honest answer is 'both'. A common target looks like this: an integration or API-management layer sits at the edge of the old systems and exposes stable APIs; new products are built as microservices behind an API gateway and talk to each other through events.

If you are moving away from a single big application or an old SOA estate, do it step by step using the 'strangler fig' pattern. Pick one capability, route its traffic to a new service, put a translation layer between the old and new data models, and retire the old path once traffic has drained. Repeat. Where an ESB exists, slowly narrow its job to protocol translation for systems that cannot change, and stop adding new business logic to it.

7. Common Mistakes and How to Avoid Them

Mistake	What it looks like and what to do
Distributed monolith	Code is split into services, but they share a database, call each other in long chains, or must be released together. Fix: one database per service, events instead of direct calls, contract tests.
Business logic in the hub	Rules and processes pile up inside the ESB until nobody dares change it. Fix: keep the hub for translation and routing only; put logic in services.
Nanoservices	Services so tiny that network delays and deployment coordination dominate. Fix: size services to a business area and a team.
Ignoring Conway's Law	Adopting microservices without changing team structure. You get the old silos with more parts. Fix: line up teams with services.
Weak observability	Trying to debug across dozens of processes without tracing, correlation IDs or central logs. Fix: build the monitoring platform before you grow the number of services.
One rigid data model	In SOA, a single company-wide 'Customer' schema becomes a change bottleneck. Fix: let each area keep its own model and translate at the boundaries.

8. Conclusion

SOA and Microservices answer different questions. SOA asks: how can all the systems in a company work together in a consistent way? Microservices ask: how can a product team change its software quickly and safely? New product development has clearly moved towards microservices, helped by

Kubernetes, service mesh and mature frameworks. Integration platforms have not gone away; they are reinventing themselves around APIs rather than the classic ESB.

The architect's job is to work out which question is really being asked, to be honest about how ready the organisation and its operations are, and to resist splitting the software before the business domain and the team are ready for it.

Glossary

Term	Simple meaning
API gateway	A single front door that receives outside requests and passes them to the right service.
Bounded context	One clear business area with its own vocabulary and data, such as 'Billing' or 'Shipping'.
Circuit breaker	A safety switch in code. If a service keeps failing, stop calling it for a while so the failure does not spread.
Container	A light, self-contained package that holds a program and everything it needs to run.
Conway's Law	Software ends up shaped like the organisation that built it.
ESB (Enterprise Service Bus)	The central hub in SOA that translates, routes and controls messages between systems.
Eventual consistency	Data in different services may differ for a short time but becomes the same soon after.
Kubernetes	Software that runs and manages many containers across many machines.
Monolith	One large program deployed as a single unit.
Saga	A long business action split into steps, each with an 'undo' step in case a later step fails.
Service mesh	A layer of helper processes that handle security, retries and monitoring between services (for example Istio, Linkerd).
Strangler fig	A way to replace an old system piece by piece, named after a vine that slowly grows around and replaces a tree.